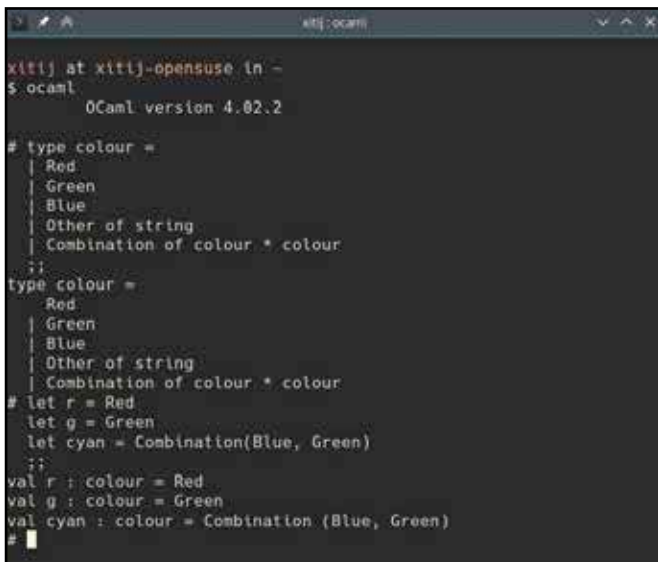




```

xiti@xiti-opensuse in ~
$ ocaml
OCaml version 4.02.2

# type colour =
| Red
| Green
| Blue
| Other of string
| Combination of colour * colour
;;
type colour =
| Red
| Green
| Blue
| Other of string
| Combination of colour * colour
# let r = Red
let g = Green
let cyan = Combination(Blue, Green)
;;
val r : colour = Red
val g : colour = Green
val cyan : colour = Combination (Blue, Green)
#

```

With minor changes (just remove capitals from variable names in some cases) you can have most of the code from this article running in OCaml.



```

xiti@xiti-opensuse in ~
$ fsharp
F# Interactive for F# 4.0 (Open Source Edition)
Freely distributed under the Apache 2.0 Open Source License.
For help type #help;;

> let x = 11;;
val x : int = 11

> x = x + 1;;
val it : bool = false
> x <- x + 1;;
x <- x + 1;;

/home/xiti/.xfs0in(3,3): error FS0027: This value is not mutable
>

```

F# Interactive is a great way to get started with F# code. Always remember to type ';;' after a block of code to tell the interpreter to execute it.

tions are called higher order functions) and they are an important part of functional programming. Functional languages also favour immutability and functional purity.

What is Immutability

While in most object-oriented and imperative languages when you constantly change the values of variables to perform computations, functional languages go for a more mathematical route, once you define a value it can't change.

Let's see this in action, by getting things wrong. Open the F# interactive console and type the following:

```
let x = 11;;
```

Note the double semi-colon at the end of the line. You do not usually need this while coding in a text file, however while working in the interactive interpreter it is required to tell the F# interpreter where an expression ends. You can print the value of x at any time by simply typing `x;;` at this prompt.

Now 11 is a rather dull number, let's try to set x to something cool, like 42. In most languages this would be as simple as setting x to 42 or adding 31 to it. Let's try both:

```
x = 42;;
```

```
x = x + 31;;
```

In both cases you should see output like:

```
val it : bool = false
```

This is probably not what you were expecting. In F# the above two lines are actually checking if x is equal to 42 or if x is equal to x + 31, both are obviously false. This is equivalent to `x == 42` or `x == x + 31` in languages like Java, C++ etc.

In fact, since 'variables' don't really vary in F# they are generally called identifiers, and by using them, you are said to be binding a value to an identifier.

There are ways to change the value of a variable in F# using the syntax `x <- 42`, however, even that will fail in this case since x is immutable by default. In fact try it right now and see F# complain.

It is possible to declare x such that it can be modified but we won't go into that here, that's not what functional programming is about. But you are free to explore that on your own.

What are Pure Functions

Pure functions are another core principle of functional programming languages. While most functional programming languages allow for some 'impure' functions, they highly emphasise writing functions that are pure.

A pure function is one simply takes its parameters and produces a single output. It is not affected by anything outside the function, and it affects nothing outside the function.

This is similar to how mathematical functions work. You don't expect the sine of a value to change when you calculate it multiple times, nor do you expect the value of the sine function to depend on the previous value of sine.

Pure functions don't use or change global variables or the parameters they receive. Technically, multiple calls to a traditional printf function will produce output in different lines, thus its output is dependent on previous calls.

Similarly date / time / random number generator functions are impure because they return a different output each time. So obviously its impossible to be completely pure without entirely crippling your software.

If you're wondering why it's advantageous to put these restrictions on yourself, know that such a design makes it very easy to parallelise your applications and make them multi-threaded.

Functions

Since we are looking at a functional language, it's only fair that we start by looking at functions.

You'll be pleased to know that creating a function is very simple:

```
let sum a b =
```

```
  a + b
```

Indentation matters in F# just like in Python, although you could have written the above in a single line if you wanted. Remember that in the interpreter you need to type `;;` after typing some code to evaluate it, simply pressing return isn't enough.

A core facet of functions in F# is currying, or partial application of functions. This allows you to provide the parameters to a function in parts rather in one go.

For instance, if a function needs four parameters, you can provide two of them, and rather than raise an error you will get back a function that can take the rest of the parameters.

Imagine we have a function to calculate the simple interest based on the rate and time. There are many accounts but the rate remains